NullMarket Phase 11

Spark Performance Concepts — Interview Study Guide

Based on the Phase 11 roadmap exercises and the execution plans observed in NullMarket.

Phase objective: Understand how Spark executes DataFrame workloads, where shuffles occur, how partitioning affects work, and why a genuinely small dimension can be broadcast to avoid shuffling a larger fact dataset.

1. What You Must Be Able to Explain

These are the Phase 11 completion-gate concepts. You should be able to explain them conversationally without relying on code.

Concept	What you need to know
Driver	Coordinates the Spark application: builds/optimizes plans, schedules work, and tracks execution. In cluster mode it communicates with executors; NullMarket currently runs locally with local[*].
Executor	A process in a distributed Spark application that runs tasks and can hold intermediate/cached data. In NullMarket local mode, the work stays on one machine, but the distributed execution model is the one you must understand.
Partition	A chunk of a DataFrame/RDD that becomes a unit of parallel work. Within a stage, Spark generally runs one task per partition.
Job	Execution triggered by an action. An action may result in one or more Spark jobs depending on the plan/runtime behavior.
Stage	A group of tasks that can run without crossing a shuffle boundary. Shuffles typically divide the DAG into stages.
Task	The smallest scheduled unit of Spark execution. A task processes one partition for one stage.
Lazy evaluation	Transformations build lineage/plans rather than immediately materializing results. Actions such as count(), show(), collect(), or writes trigger execution.
DAG	The directed acyclic graph representing dependencies among Spark operations. Spark optimizes the work before execution.
Shuffle	Redistribution of records across partitions, normally by key or ordering requirement. It can involve serialization, network/disk I/O, sorting, and additional stages.

2. Spark Execution Model

```
Transformation chain
        ↓
Lazy logical plan / lineage (DAG)
        ↓
Action triggers execution
        ↓
Job
        ↓
Stages (often separated by shuffles)
        ↓
Tasks
        ↓
Each task processes a partition
```

Interview phrasing: An action triggers Spark execution. Spark executes the job as stages, with shuffle boundaries commonly separating stages. Within each stage, tasks process the available partitions.

3. Reading df.explain()

`df.explain()` lets you inspect how Spark understands and intends to execute a DataFrame computation. In Phase 11 you inspected real NullMarket filters, joins, aggregations, and windows.

Concept	What you need to know
Parsed logical plan	Spark has parsed the expressions and operations you wrote.
Analyzed logical plan	Spark has resolved columns, types, and relationships against the available schema.
Optimized logical plan	Catalyst has simplified/rearranged the logical work where possible.
Physical plan	Concrete operators Spark intends to execute: scans, joins, aggregates, sorts, exchanges, windows, broadcasts, etc.

Plan markers worth recognizing

Concept	What you need to know
Exchange	Data is being redistributed. In these Phase 11 plans, this was the clearest shuffle indicator.
HashAggregate	Hash-based aggregation. You observed partial aggregation before an Exchange and final aggregation after it.
Sort	Rows must be ordered for an operation such as orderBy(), a sort-merge join, or an ordered window.
BroadcastExchange	Spark is preparing a small relation to be copied for a broadcast join.
BroadcastHashJoin	A hash join where the small side is broadcast; the larger side avoids a join-key shuffle.
SortMergeJoin	Both sides are generally repartitioned/sorted on the join key before merging.
SinglePartition	Relevant rows are being brought into one partition, as occurred for NullMarket global windows.
PushedFilters	A data-source filter was pushed toward Parquet so irrelevant data can be avoided earlier.
PartitionFilters	Spark can use the physical partition directory values to prune irrelevant partitions.

4. Narrow vs. Wide Transformations

The distinction is about partition dependencies—not merely the function name.

Narrow	Wide
Each output partition can be computed from a single input partition; no cross-partition redistribution is required.	An output partition depends on records that may come from multiple input partitions, so data must usually be redistributed.
NullMarket examples: select, filter, withColumn.	NullMarket examples: groupBy aggregation, global orderBy/windows, repartition, and shuffle-based joins.

Observed evidence: the filter/select/withColumn chain had no Exchange. Product aggregation contained `Exchange hashpartitioning(product_key, 200)`, proving that rows had to be redistributed before the final aggregation.

Engineering rule: avoid unnecessary shuffles, not all shuffles. Some business calculations fundamentally require records with the same key or global order to meet.

5. DataFrame Partitioning

Phase 11 distinguished in-memory Spark partitions from the physical Parquet directory partitioning created in Phase 10.

Concept	What you need to know
Too few partitions	Can underutilize available CPU/executors, produce very large tasks, and reduce parallelism.
Too many partitions	Can create excessive scheduling overhead, tiny tasks, and—when written—many small files.
Goal	Use enough reasonably sized partitions to exploit available compute without excessive task/file overhead. There is no universally correct partition count.

repartition() vs coalesce()

Concept	What you need to know
repartition()	Can increase or decrease partition count and optionally partition by key. It performs redistribution and normally causes a full shuffle.
coalesce()	Primarily reduces partition count without a full redistribution. It is cheaper, but resulting partitions can be uneven.

NullMarket experiment: `fact_sales` read as 1 partition; `repartition(8, "product_key")` produced 8 partitions and an Exchange; `coalesce(2)` then reduced that lineage to 2 partitions using a Coalesce operator. The earlier Exchange remained visible because the input to coalesce had already been repartitioned.

Important: the observed 200 shuffle partitions in some aggregation plans were Spark shuffle partitioning behavior, not evidence that the original `fact_sales` input had 200 partitions.

6. Broadcast Joins

A broadcast join is appropriate when one side is genuinely small enough to copy to every executor. That allows the larger side to remain where it is instead of being shuffled by the join key.

```
Shuffle-based baseline
large fact ── Exchange/Sort ──┐
                              ├─ SortMergeJoin
small dim  ── Exchange/Sort ──┘

Broadcast strategy
large fact ──────────────────┐
                              ├─ BroadcastHashJoin
small dim  ── Broadcast ─────┘
```

NullMarket evidence: with auto-broadcast disabled, the `fact_sales` → `dim_product` join used two hash-partitioning Exchanges and a SortMergeJoin. With an explicit broadcast hint, the plan used BroadcastExchange + BroadcastHashJoin, and the large fact side did not receive a join-key Exchange.

Safety condition: do not broadcast a table merely because it is named a dimension. If it is too large, every executor must hold a copy, creating memory pressure, garbage collection, or failures.

Do not claim a measured speedup: Phase 11 compared execution plans, not benchmarked runtime improvements.

7. Data Skew

Data skew occurs when some keys contain disproportionately more rows than others. After a key-based shuffle, those heavy keys create oversized partitions and a few slow tasks can become stage stragglers.

```
Even distribution:   100 | 102 | 98 | 101   → similar task sizes
Skewed distribution:  100 | 103 | 99 | 10,000 → one straggler task
```

NullMarket inspection: the most frequent observed `product_key` had 28 fact rows, with several others around 20–25. That is enough to demonstrate how to inspect key distribution, but it does not establish a material skew problem in this small synthetic dataset.

8. Window Functions and the Single-Partition Warning

You observed the warning: `WindowExec: No Partition Defined for Window operation! Moving all data to a single partition.` This is not a data-quality error.

```
Global window
Window.orderBy("net_sales")
→ every row participates in one ordered group
→ Exchange SinglePartition

Partitioned window
Window.partitionBy("province").orderBy("net_sales")
→ independent ordered groups by province
→ rows can remain distributed by group after any required shuffle
```

In NullMarket, global product ranking and the chronological rolling-revenue calculation intentionally use global `Window.orderBy(...)`. Spark therefore needs one globally ordered sequence. That is defensible for the business calculation, but it can become a scalability bottleneck on a very large dataset.

A partitioned window can remain distributed by group, although Spark may still need to shuffle first so all rows for each partition key are colocated.

9. What NullMarket Actually Demonstrated

Experiment	Observed result	Meaning
Initial partitions	dim_product=1, dim_date=1, fact_sales=1, inventory fact=7	Small demo outputs; inventory reflects its seven date-key partitions on read.
Narrow chain	No Exchange	filter/select/withColumn did not require row redistribution.
Parquet filter	PushedFilters on net_sales	Predicate pushdown was visible in the scan plan.
Inventory date filter	PartitionFilters: date_key=20260101	Spark could prune the Phase 10 partition directory layout.
Normal product join	BroadcastHashJoin	Spark automatically recognized dim_product as small enough to broadcast.
Product groupBy	HashAggregate → Exchange → HashAggregate	Partial aggregation followed by shuffle and final aggregation.
Global ranking/trend windows	Exchange SinglePartition → Sort → Window	Global ordering required all relevant rows in one ordered partition.
repartition	hashpartitioning(product_key, 8)	Full redistribution to eight partitions.
coalesce	Coalesce(2)	Reduced partition count without another full redistribution.
Broadcast disabled join	Two Exchanges + SortMergeJoin	Both sides were redistributed/sorted by product_key.
Explicit broadcast join	BroadcastExchange + BroadcastHashJoin	Small dimension broadcast; no join-key shuffle on the fact side.
Skew inspection	Top product count=28	Useful diagnostic example; not evidence of a significant skew problem.

10. Implementation vs. Understanding

Concept	What you need to know
Implemented in NullMarket	Execution-plan inspection; partition-count inspection; repartition/coalesce experiments; automatic and explicit broadcast joins; aggregation/window shuffle inspection; basic key-frequency skew inspection.
Must understand and explain	Driver/executor roles; lazy evaluation; DAG/job/stage/task/partition hierarchy; why shuffles occur and cost more; narrow vs wide dependencies; partition-count tradeoffs; broadcast safety; skew effects; global-window warning.
Recognize conceptually	`AdaptiveSparkPlan` indicates Spark Adaptive Query Execution is involved in physical planning/runtime adaptation. Phase 11 did not require you to tune AQE.
Not claimed	No enterprise scale, performance speedup, cost reduction, or optimal partition count was measured in Phase 11.

11. Interview-Ready Answers

What does df.explain() tell you?

It exposes Spark's logical and physical execution plans. I use the physical plan to identify scans, filters, joins, aggregation strategies, sorts, broadcasts, windows, and Exchange operators that indicate data redistribution.

What is a shuffle?

A shuffle redistributes records across partitions, usually by a key or ordering requirement. It is relatively expensive because it can require serialization, network/disk I/O, sorting, and additional stages.

Narrow vs wide?

Narrow transformations can process each partition independently; wide transformations need data from multiple input partitions to be brought together, usually causing a shuffle.

repartition vs coalesce?

repartition can increase or decrease partitions and redistributes data through a shuffle; coalesce is mainly for reducing partitions without a full redistribution, though it can leave uneven partitions.

Why broadcast a dimension?

If a dimension is genuinely small, Spark can copy it to each executor so the much larger fact side does not need a join-key shuffle. I would not force broadcast if the table is too large for safe executor memory.

What is data skew?

Skew means some keys own disproportionately more rows. After a key-based shuffle, those keys create large partitions and straggler tasks that can delay the entire stage.

What did the WindowExec warning mean?

Our global windows had orderBy but no partitionBy, so Spark needed one globally ordered group and moved all relevant rows to a single partition. It is correct for those global calculations but can become a scalability bottleneck.

12. Phase 11 Final Checklist

- I can explain driver vs executor, including the fact that NullMarket currently runs in local[*] rather than a real multi-node cluster.
- I can explain job → stage → task → partition and how shuffle boundaries relate to stages.
- I can explain lazy evaluation and name common actions.
- I can identify Exchange, BroadcastHashJoin, SortMergeJoin, HashAggregate, Sort, Window, PushedFilters, and PartitionFilters in an execution plan.
- I can explain why a groupBy or global ordering/window can require a shuffle.
- I can explain repartition() vs coalesce() and why partition count is a tradeoff.
- I can explain why a small broadcast side avoids shuffling the large side and when broadcasting is unsafe.
- I can explain data skew and why one heavy key can create a slow task.
- I can explain the NullMarket WindowExec warning without calling it an error.
- I will not claim performance improvements that Phase 11 did not benchmark.